

Enterprise Ansible Deployment and Malware Remediation Tool

An Infrastructure as Code Platform for Proxmox VE, Controlled Malware Testing, and Automated Remediation

Authors	Connor, [Partner1-Redacted], [Partner1-Redacted]
Advisor	Prof. Furkan Paligu
Department	Computer & Digital Forensics, Division of IT & Sciences
Institution	Champlain College

Abstract

Malware remediation in enterprise and educational environments is frequently characterized by slow manual workflows, inconsistent artifact removal, and a lack of repeatable documentation. This project presents an Infrastructure as Code platform built on open-source Proxmox VE and automated through Ansible & Semaphore, designed to address these shortcomings through automated VM deployment, controlled malware execution, and automated detection and rollback remediation.

A three-node Proxmox cluster was constructed and eleven production-ready VM templates were built across Windows, Linux, and network OS families. The Ansible deployment pipeline reduced 10-VM full deployment time from a manual average of approximately 20 minutes to 2 minutes and 17 seconds, an 8.5x improvement, with near-zero variability between runs. Malware execution was isolated within a dedicated FakeNet-NG network segment with no external connectivity. Three malware families were selected: WannaCry (ransomware), LokiBot (infostealer), and Kovter (fileless clickfraud). A generalized behavioral detection script was developed in PowerShell that scores each VM against weighted indicators across six categories: malware-specific IOCs, filesystem, registry, process, PowerShell Script Block logs, and Windows event logs, without relying on malware signatures. An Ansible orchestration layer stages and executes this script via the QEMU guest agent channel, parses the scored JSON output, and automatically rolls back confirmed infections to a verified clean VM snapshot.

The remediation pipeline was validated against all three malware families and benchmarked against a manual remediation baseline. Manual remediation of the three infected VMs required approximately 39 minutes with no artifact documentation produced. A sequential automated pipeline completed the same three VMs in 8 minutes and 27 seconds with a full structured JSON report generated automatically. A parallel iteration reduced total duration to 6 minutes and 50 seconds, with a 5.7x improvement over the manual baseline. All three malware families were properly detected and rolled back on every run with zero false positive rollbacks on clean VMs.

1. Introduction

Malware incidents in enterprise and educational environments continue to increase in frequency and complexity. The conventional response being manual investigation, artifact identification, and system cleanup is slow, prone to human error, and difficult to repeat consistently. Mean time to remediation in manually managed environments frequently spans multiple days, and artifact removal completeness is estimated at approximately 70% under manual workflows. The operational cost of this approach scales poorly as the number of affected systems grows.

Beyond remediation, the manual deployment of virtual machine environments presents its own operational burden. Provisioning a 10-VM enterprise environment manually requires an analyst to individually clone each template, configure network settings, assign IP addresses, and register VMs in the appropriate resource pools, a process that is both time-consuming and prone to configuration drift between runs. Automating this process is a prerequisite for building a repeatable, auditable malware testing platform.

Simultaneously, the enterprise virtualization landscape has undergone a significant shift. The acquisition of VMware by Broadcom resulted in estimated price increases of 60 to 80 percent for existing customers, prompting widespread evaluation of open-source alternatives. Proxmox VE, a Debian-based hypervisor supporting both KVM virtual machines and LXC containers, has emerged as the primary open-source alternative under consideration by many organizations.

This project addresses all of these problems simultaneously. An Infrastructure as Code platform was constructed on a three-node Proxmox VE cluster, automated through Ansible playbooks, and used to execute controlled malware tests against isolated Windows VMs. The platform automates the full lifecycle: environment deployment, malware isolation, behavioral detection, and snapshot rollback, all without operator intervention after the initial command is issued.

The primary research questions are whether automated Ansible-based VM deployment can outperform manual deployment in speed and consistency, and whether automated behavioral detection and rollback can meaningfully outperform manual remediation across multiple malware families simultaneously. Three malware families were selected to represent distinct behavioral

profiles: WannaCry (ransomware/worm), LokiBot (persistent infostealer), and Kovter (fileless clickfraud). The platform was validated against all three and benchmarked against manual baselines for both deployment and remediation.

2. Literature Review

The project drew on industry tools including Terraform, Pulumi, Puppet, and Red Hat Satellite to inform a custom script suite using pvesh commands, embedded Python, and the qemu-agent for agentless compliance and malware remediation. Frameworks such as ISO/IEC 27035, MITRE ATT&CK, NIST CSF 2.0, and CIS structured the scripts across incident lifecycle phases, enabling SIEM-based detection, Semaphore-driven response, and automated logging. Supporting academic literature validated the use of Ansible for idempotent environment management, Proxmox for secure resource-constrained virtualization, and informed the project's approach to fileless malware detection and IaC scripting practices.

2.1 Main Inspirations

Terraform	Inspired dynamic VM discovery
Pulumni	Inspired python usage within scripts
Puppet	Guided Compliance and Malware Remediation Goals
Zabbix/Nagios	Inspired the focus of remote remediation [QEMU agent]
Vagrant	This inspired VM Deployment patterns [Reducing time]
ISO/IEC	[PIARL] Plan -> Identify -> Assess -> Respond -> Learn
MITRE ATT&CK	Used to research and map Malware strain identity.
NIST CSF 2.0	Inspired VM hash collection and integrity verification
CIS	Used for VM Hardening standards; Automatable
Ansible in University Labs	Used to gauge and build around Idempotency [consistent configuration for environmental recreation]
Proxmox VE Academic Cloud	Shows an active use case for our project and inspired environmental setup
Fileless Malware Survey	Informed rule-based & Behavioral detection methods
IAC Technologies Analysis	Guided the bases for our IAC tool creation [language choice]

3. Methodology

3.1 Infrastructure Construction

Proxmox Node 3, Sam [Cisco UCS C250 M1, 24 CPUs, 70.70 GiB RAM], was the target for our malware testing, benchmarks & Deployments to maintain consistency. A total of Eleven VM templates, hosted on a 4TB NFS share, were used in the deployments. CIS Hardened Windows, Windows Server 2019, Linux/Rhel Distributions, VyOS & pfSense VM templates were used. Each VM has QEMU Guest Agent enabled. A SHA256 Manifest was created for each template. The Ansible deployment pipeline, originally sequential per-user, was rebuilt as a parallel architecture with pre-computed VMIDs, throttled cloning, and idempotent re-entry guards, with post-deployment IP configuration handled via the QEMU Guest Agent.

3.2 Malware Isolation

Malware execution was contained within a dedicated isolated network segment. FakeNet-NG was deployed to simulate WAN services within this segment. All 7-zipped samples were sourced from verified repositories. Samples selected include: WannaCry (MITRE T1486, T1021.002, T1543), LokiBot (T1547.001, T1056, T1005), and Kovter (T1547.001, T1053, T1056). Each malware VM was snapshotted twice: once after provisioning with logging configured and event logs cleared (pre-malware-421), and once immediately after detonation. The pre-malware-421 snapshot serves as the rollback target for all remediation runs.

3.3 Detection Pipeline

3.3.1 Communication Architecture

All Controllers to VM Communication use the QEMU guest agent channel exclusively; allowing for isolated VM management. Commands being issued are done via ``qm guest exec`` and ``pvesh API`` calls. The detection PS1 is staged into each VM via a Python helper that base64-encodes the script, splits it into 8000-character chunks, and writes each chunk sequentially via PowerShell Set-Content and Add-Content commands through the agent. Once staged, the script executes and returns a compressed JSON object as stdout.

3.3.2 Weighted Scoring System

The detection script evaluates six indicator categories [malware-specific IOC checks having the highest weights] WannaCry file artifacts weight 12–15, LokiBot process pattern matching weights 15, and Kovter PowerShell stager and fileless scriptblock patterns weight 10–14, where a single IOC hit immediately confirms infection. Generic behavioral indicators span filesystem (user-writable executables weight 3, high-entropy filenames weight 2, ransom note patterns weight 3), registry (Run key persistence weight 5, Winlogon hijacking weight 4, suspicious scheduled tasks weight 5), process (user-writable path weight 4, unusual PowerShell parent weight 4), PowerShell Script Block logs (IEX/DownloadString/FromBase64String weight 3 each, vssadmin delete weight 5, burst activity weight 2), and Windows event logs (Event 7045 weight 3, Event 4698 weight 3, Event 1102 weight 5, Defender detection weight 10); absent a direct IOC match, confirmed infection requires a cumulative score of 12 or higher, at least two strong indicators, signals from at least two categories, and at least one non-PowerShell signal, or alternatively a PowerShell stager pattern within the last 15 minutes with a score of 5 or above. Three false positives were identified and mitigated. Those being Windows Crypto API random-looking key filenames, Defender's ProgramData process path, and the detection script's own PowerShell execution self-flagging. Following this clean VMs consistently scored below the infection threshold.

3.4 Parallel Pipeline Architecture

The initial pipeline processed VMs sequentially. A second iteration restructured scan.yml into three plays: play 1 (localhost) handles setup and registers one Ansible host per VMID and play 2 uses Ansible's free strategy to execute per_vm_scan.yml against all registered hosts simultaneously; Play 3 merges per-VM result files and writes the final report. Each VM writes its result to /tmp/vm_result_<vmid>.json independently rather than appending to a shared variable, avoiding race conditions from concurrent execution.

3.5 Benchmarking

The deployment benchmark measured VM cloning time and full deployment time (cloning, powering on, IP assignment, and pool creation) across manual and automated runs for a 10-VM scenario. The remediation benchmark measured detection and roll back times for all three infected VMs comparatively between the manual, automated sequential, and parallel automated run. Manual deployment runs were performed via the Proxmox web UI. Manual remediation was performed using Task Manager, Registry Editor, Event Viewer, and the PowerShell Operational log.

4. Results

4.1 Deployment Benchmarking

The deployment pipeline was benchmarked across two scenarios. The first measured VM cloning time only across five manual and five automated runs. The second measured full deployment time such as cloning, powering on, static IP assignment, and pool creation, across three manual and three automated runs.

4.1.1 Cloning Benchmark (10 VMs)

Method	Runs	Avg Time	Std Dev	Speedup
Manual	5	2 min 26 sec	34.0s	–
Ansible	5	1 min 12 sec	0.4s	2.02x faster

Manual cloning required individually configuring each VM's target node, VMID, name, storage, and clone type in the Proxmox UI. Human variability produced a standard deviation of 34 seconds across five runs. The Ansible script produced identical results across all five runs with a standard deviation of 0.4 seconds, demonstrating not only speed improvement but near-perfect repeatability.

4.1.2 Full Deployment Benchmark (10 VMs)

Method	Run	Avg Time
Manual	1	22 min 17 sec
Manual	2	20 min 26 sec
Manual	3	17 min 3 sec
Ansible	4	2 min 17 sec
Ansible	5	2 min 17 sec
Ansible	6	2 min 17 sec

The full deployment benchmark includes VM cloning, powering on all VMs, assigning static IP addresses via the QEMU guest agent, and creating Proxmox pools with VM membership. Manual execution required between 17 and 22 minutes across three runs, with variability driven by the sequential nature of each step and human reaction time. The Ansible pipeline completed all steps in 2 minutes and 17 seconds on all three runs, approximately 8.5x faster than the manual average with zero variability between runs.

4.2 Detection Accuracy

All three malware families were correctly detected and confirmed as infected on every pipeline run with high confidence. The following table summarizes outcomes.

VM	Score	Confidence	Decision Reason
WannaCry Infected VM	50	High	wannacry_ioc_detected
LokiBot Infected VM	17	High	lokibot_ioc_detected
Kovter Infected VM	23	High	kovter_ioc_detected

4.3 False Positive Rate

Clean VMs restored to the pre-malware snapshot were tested against the pipeline after tuning. Zero false positive rollbacks were observed. Clean VMs scored between 0 and 4 after exclusion of the Crypto API path, Defender process path, and self-detection markers, well below the confirmed infection threshold of 12.

4.4 Remediation Benchmarking

Metric	Manual	Ansible Iteration 1	Ansible Iteration 2
Total duration	~39 minutes	8 min 27 sec	6 min 50 sec
WannaCry time	~13 minutes	~2 min 15 sec	~2 minutes
LokiBot time	~8 minutes	~2 minutes	~2 minutes
Kovter time	~16 minutes	~4 minutes	~4 minutes
All 3 detected + rolled back	Yes	Yes	Yes
Operator actions required	Continuous	0	0
Report generated	No – manual notes	Yes – automated JSON report	Yes – automated JSON report

The sequential automated pipeline ran each VM in turn: WannaCry completing before LokiBot began, LokiBot completing before Kovter began. The parallel pipeline launched all three

simultaneously using Ansible's free strategy, with each VM executing the full detection, scoring, and rollback workflow concurrently. Both automated runs produced identical structured JSON reports at `/tmp/remediation_report_<timestamp>.json` on the Ansible controller, containing per-VM fields including `scan_started_at`, `detected_at`, `nuked_at`, `redeployed_at`, detection score, confidence, decision reason, malware family, and the full list of indicators with category, detail, and weight.

5. Discussion

5.1 Interpretation of Results

Both the deployment and remediation benchmarks demonstrate the same underlying pattern: automation eliminates human variability entirely while compressing execution time by a factor of 5 to 9. The deployment benchmark is particularly striking, with a standard deviation of 0.4 seconds across five automated runs compared to 34 seconds for manual runs illustrates that consistency, not just speed, is a primary advantage of the automated approach. At scale, this consistency becomes critical: a deployment that varies by 34 seconds per VM across 30 VMs introduces nearly 17 minutes of unpredictable timing, whereas the automated pipeline is deterministic regardless of scale.

The remediation results show a similar pattern. The parallel pipeline's improvement over the sequential pipeline (approximately 19%) is smaller than might be expected from concurrent execution. This is because parallelism is bounded by the slowest VM. Kovter consistently required approximately 4 minutes in both automated runs. Further gains would require reducing per-VM scan time rather than adding concurrency. The detection approach, behavioral scoring rather than signature matching, proved essential for detecting Kovter, which is entirely fileless. PowerShell Script Block log correlation detected it where file-hash detection would have failed entirely. Similarly, LokiBot's self-deleting dropper meant filesystem scanning alone would have missed it. Detection relied on catching the runtime process in memory. This validates the multi-category scoring approach over any single-category detection strategy.

5.2 Limitations

WannaCry presents a time-sensitivity constraint not present in the other families. Because it actively encrypts files after detonation, the VM state degrades over time and PowerShell execution eventually becomes unavailable as system files are affected. Detection must occur within approximately 15 minutes of detonation in the lab environment. LokiBot's dead C2 endpoint also likely affected its execution path, the sample targets an endpoint unreachable in the isolated lab, meaning some persistence behaviors that require a successful C2 check-in may not have manifested.

5.3 Conclusion

This project demonstrates that automated Ansible-based VM deployment and malware detection and rollback are viable and significantly superior alternatives to manual processes across speed, consistency, and documentation quality. Deployment time was reduced 8.5x with zero variability between runs. Remediation time was reduced 5.7x while simultaneously producing richer artifact documentation than any manual process could generate at scale. The platform correctly identified all three malware families, including a fileless sample undetectable by file-based methods, and performed automated rollback with zero false positives on clean VMs. The open-source stack (Proxmox VE, Ansible, Python, PowerShell) reproduces capabilities comparable to commercial enterprise EDR and deployment platforms at effectively zero licensing cost, validating its suitability as a VMware replacement candidate for educational and small enterprise environments.